

Strata Protocol (\$STRAT): Architecture Diagram & Requirements Document

Team 13 — Strata Protocol

Protocol Focus: Dual-Vault RWA Yield & Thematic Basket Protocol on Solana

Target Region: Non-US (Reg S Compliant)

Authors: Ankit Tiwari, Viraz, Gideøn

1. Protocol Overview & Key Actors

Strata Protocol introduces a two-tier vault architecture designed to transform static, isolated tokenized real-world assets into composable, yield-bearing building blocks on Solana:

1. **Vault 1 (xau_carry_vault — REQ01 MVP Scope):** A delta-neutral strategy vault where depositors supply USDC and receive a 1:1 receipt token (\$vXAU). In the roadmap strategy layer, the vault executes a delta-neutral carry trade by establishing a spot gold position (PAXG) via DEX CPI paired with a short XAU perpetual futures position (GMTrade/gsol) to harvest funding rate yield.
2. **Vault 2 (basket_weighted_vault — REQ02 & REQ03 MVP Scope):** A thematic RWA index engine that accepts a multi-asset recipe (vXAU, PAXG, USDY, SPYx). Users deposit component assets to mint composite \$BASKET tokens (\$STRAT-INDEX). Permissionless arbitrageurs keep portfolio asset weights aligned with target parameters via an oracle-priced atomic swap mechanism using Pyth Network price feeds (PriceUpdateV2).

Actor Categories

- **Direct Actors (Signers):**
 - **Depositor:** Signs deposit_usdc_v1 (REQ01) and deposit_to_basket_v2 (REQ02).
 - **Rebalancer / Arbitrageur:** Signs rebalance_basket (REQ03) to swap underweight tokens for overweight tokens at an oracle-backed discount.
- **Beneficiaries:** Token holders accruing yield through proportional basket share appreciation and net funding carry.
- **Administrators (Signers):** Protocol Admin / Governance multisig executing setup prerequisites (initialize_vault_v1, initialize_vault_v2), configuring target asset weights (in basis points), rebalance bands, discounts, and Pyth oracle feed IDs.
- **Stakeholders:** External protocol dependencies including Pyth Network (PriceUpdateV2), SPL Token Program, SPL Token-2022 Program, Associated Token Program, and System Program.

2. Atomic Protocol Requirements & Core Use Cases

Each core use case represents **exactly one atomic state transition** mapped to **one Anchor instruction handler**.

REQ01: Deposit USDC into Carry Vault (**deposit_usdc_v1**)

- **Use Case:** Depositor supplies a positive amount of USDC into Vault 1 and receives a 1:1 receipt token (**\$vXAU**).
- **Atomic State Transition:** User USDC ATA → **vault_v1_usdc_account** (PDA authority); **vxau_mint** (PDA authority) mints **\$vXAU** → User **vXAU** ATA.
- **Anchor Handler:** **xau_carry_vault::deposit_usdc_v1(amount: u64)**
- **Signer:** Depositor (User Wallet).

REQ02: Deposit 4-Asset Basket (**deposit_to_basket_v2**)

- **Use Case:** Depositor supplies the required quantities of a 4-asset weighted basket (**vXAU**, **PAXG**, **USDY**, **SPYx**) to mint composite **\$BASKET** tokens.
- **Atomic State Transition:** Component tokens transferred from User ATAs → Vault 2 asset accounts (**vault_v2_usdc_account**, **vault_v2_asset_accounts**); **basket_mint** mints **\$BASKET** → User Basket ATA (calculated pro-rata: $\text{basket_out} = \text{supply} * \min(\text{amount_i} / \text{bal_i})$).
- **Anchor Handler:** **basket_weighted_vault::deposit_to_basket_v2(amounts: [u64; 4])**
- **Signer:** Depositor (User Wallet).

REQ03: Permissionless Basket Rebalance (**rebalance_basket**)

- **Use Case:** Rebalancer/Arbitrageur provides an underweight component token (**token_in**) and receives an overweight component token (**token_out**) from Vault 2 at an oracle-priced discount to bring asset weights back toward target.
- **Atomic State Transition:** Rebalancer ATA (**token_in**) → Vault 2 Asset Account (**token_in**); Vault 2 Asset Account (**token_out**) → Rebalancer ATA (**token_out**).
- **Anchor Handler:** **basket_weighted_vault::rebalance_basket(token_in, token_out, amount_in: u64, min_out: u64)**
- **Signer:** Rebalancer / Keeper / Arbitrageur.

3. Granularity Self-Check & Adversarial Analysis

3.1 Manual Granularity Self-Check (Pass 1)

1. **Atomicity (PASS):** **deposit_usdc_v1** (REQ01), **deposit_to_basket_v2** (REQ02), and **rebalance_basket** (REQ03) each execute as exactly one atomic instruction handler.

2. **State Ownership (PASS):** Program owns `vault_v1_state` and `vault_v2_state`. Token accounts and mints are owned by SPL Token / Token-2022 Programs with PDA authorities.
3. **Real Signers (PASS):** User explicitly signs deposits (`user: Signer<'info>`); rebalancer explicitly signs rebalance (`rebalancer: Signer<'info>`).
4. **On-Chain vs. Client-Side (PASS):** On-chain code strictly executes token transfers, Pyth oracle pricing, and minting. UI weight displays and trade sizing calculations are client-side.

3.2 Adversarial Analysis (Pass 2 — Red-Team AI Evaluation)

Accepted & Remediated Findings

1. **A1 (REQ01 Share Semantics):** Explicitly defined 1:1 receipt token semantics (`vXAU`) for the POC deposit scope, noting that NAV-based dynamic share valuation applies to the live strategy layer.
2. **A2 (REQ02 Mint Formula):** Defined precise pro-rata minting math (`basket_out = supply * min(amount_i / bal_i)`) to prevent share dilution across varying token decimals and initial balances.
3. **A3 (Seed Derivation Circularity):** Fixed seed structure: `vault_v2_state` is derived from an independent `basket_id` (`["vault_v2", basket_id]`), and `basket_mint` is derived from `vault_v2_state` (`["basket_mint", vault_v2_state]`).
4. **A4 (Initialization Governance):** Removed `init_if_needed` from user deposit paths. Explicit setup prerequisites (`initialize_vault_v1`, `initialize_vault_v2`) signed by the Protocol Admin are required prior to any user interactions.
5. **A5 (Account Ownership & Authority Rigor):** Explicitly differentiated Data Account Owners (Our Custom Program) from Token Account/Mint Owners (SPL Token / Token-2022) and Token Authorities (Program PDAs).
6. **A6 (Keeper & Strategy Execution Boundaries):** Isolated off-chain keeper actions (hedging, funding collection, NAV accounting) into labeled roadmap modules outside the core POC deposit/rebalance scope.
7. **A7 (POC Scope Definition):** Clarified that the POC covers asset custody, receipt issuance, pro-rata deposit, and oracle-priced rebalancing, while perpetual shorting and secondary AMM liquidity represent future roadmap milestones.
8. **A8 (Shared Target Recipe):** Enforced a single shared target recipe per Vault 2 instance rather than user-customizable recipes, preserving BASKET share fungibility.
9. **A9 (CPI & Error Enumeration):** Fully specified CPI dependencies per instruction and explicit revert paths (zero amount, wrong mint, stale price feed, market closure, trade overshoot, slippage breach).

Rejected AI Flags

- **R1 (Split REQ01 into separate transfer & mint handlers):** REJECTED. The assignment rubric mandates 1 Use Case = 1 Instruction Handler. Splitting a deposit into separate transfer and mint handlers creates an unminted intermediate state and breaks transactional atomicity.
- **R2 (Split REQ02 into multiple transfer handlers and a mint handler):** REJECTED. Executing multiple CPIs within a single instruction handler is fully supported by Solana and Anchor, ensuring all component transfers and mints succeed or fail together atomically.
- **R3 (Require AMM accounts in POC deposit handlers):** REJECTED. Secondary AMM liquidity provision belongs to future roadmap integrations and does not belong in core POC vault custody handlers.

4. On-Chain Requirements Matrix

Instruction / Scope	Key Accounts	CPI Dependencies	Validation Rules	Revert Paths
Prerequisites: <code>initialize_vault_v1</code> <code>initialize_vault_v2</code>	<code>admin</code> (Signer) <code>vault_v1_state</code> (Owner: Program, seed: ["vault_v1", usdc_mint]) <code>vault_v2_state</code> (Owner: Program, seed: ["vault_v2", basket_id]) Token Vaults	System Program	Admin signature required Configures target weights (in BPS), rebalance bands, discount BPS, Pyth feed IDs	Non-admin signer Weights sum != 10,000 Invalid feed IDs
REQ01: <code>deposit_usdc_v1</code>	<code>user</code> (Signer) <code>vault_v1_state</code> (Owner: Program) <code>vault_v1_usdc_account</code> (Owner: SPL Token, Auth: PDA) <code>vxau_mint</code> (Owner: SPL Token, Auth: PDA) <code>user_usdc_ata</code> <code>user_vxau_ata</code>	SPL Token Program	<code>amount > 0</code> <code>user_usdc_ata.mint == usdc_mint</code> 1:1 mint ratio (<code>vxau_out = amount</code>)	<code>amount == 0</code> Wrong token mint Uninitialized state Insufficient user balance Token CPI failure
REQ02: <code>deposit_to_basket_v2</code>	<code>user</code> (Signer) <code>vault_v2_state</code> (Owner: Program) <code>vault_v2_usdc_account</code> (Owner: SPL Token, Auth: PDA) <code>vault_v2_asset_accounts</code> x3 (Owner: SPL Token / Token-2022, Auth:	SPL Token SPL Token-2022 (SPYx)	Pro-rata ratio enforcement (<code>basket_out = supply * min(amount_i / bal_i))</code>) <code>amounts[i] > 0</code> SPYx moved via <code>transfer_checked</code>	Any <code>amount == 0</code> Wrong asset mint Mismatched pro-rata ratios Token CPI failure

Instruction / Scope	Key Accounts	CPI Dependencies	Validation Rules	Revert Paths
	PDA)basket_mint (Owner: SPL Token, Auth: PDA)User ATAs			
REQ03:rebalance_basket	rebalancer (Signer)vault_v2_state (Owner: Program)vault_v2_asset_accounths (Owner: SPL Token / Token-2022, Auth: PDA)Rebalancer ATAsPyth Oracles (PAXG, USDY, SPYx)	Pyth Oracle Read (PriceUpdateV2)SPL Token / Token-2022	token_in must be underweightOracle price age <= max_ageout = amount_in * (p_in / p_out) * (1 + discount_bps)out >= min_outTrade cannot overshoot target weight band	Stale price (> max_age)SPYx feed stale during equity market closuretoken_in not underweightTrade overshoots targetout < min_outToken CPI failure

5. Strategy Management & Roadmap Use Cases (Vault 1 Strategy Layer)

While the core POC implements REQ01, REQ02, and REQ03, the production protocol roadmap defines the underlying delta-neutral gold carry management engine:

- UC-03 (open_strategy): Initializes StrategyState PDA (["strategy", vault_v1_state]).
- UC-04 (acquire_paxg): Swaps USDC for spot PAXG via DEX CPI to establish long gold exposure.
- UC-05 (open_short_xau): Opens short XAU perpetual futures position on GMTrade/Perp DEX.
- UC-06 (claim_funding): Settles accrued perp funding payments into Vault 1 collateral.
- UC-07 (rebalance_hedge): Adjusts short perp size to keep net delta within target parameters.
- UC-08 (reduce_short): Scales down short position during net user redemptions.
- UC-09 (withdraw): Depositor burns \$vXAU and redeems proportional USDC value.
- UC-10 / UC-11 (pause / unwind): Emergency circuit breakers to close positions and safeguard collateral.

6. System Architecture Diagram

Strata Protocol Architecture (Turbin3 Capstone POC)

